

Time-blind by Design: Diagnosing Temporal Traceability in Transformer Representations

Jefferson O. Silva^{1,2}

¹ Instituto de Tecnologia e Liderança, São Paulo, Brazil
`jefferson.oliveira@prof.inteli.edu.br`

² Pontifícia Universidade Católica de São Paulo, São Paulo, Brazil

Abstract. Words do not preserve fixed meanings across time. Without representations that reflect this, `token@time` queries become impossible: there is no way to ask where the meaning of *gay* stood in 1920, or how it moved by 1980, if both queries return the same vector. Yet time-conditioned transformers have rarely been tested on whether they provide an explicit vector for a word at a specified period whose neighbors reflect how the word is used then. Knowing that a representation encodes the period label is not the same as being able to query such neighborhoods and compare how they change over time, a distinction we formalize as *temporal traceability*. We introduce a controlled test suite with predefined trajectories (stable, drifting, bifurcating) to measure it directly.

Three findings emerge. Giving the model period information more than doubles the movement of a word’s neighbors relative to the non-temporal baseline. However, adding the same period vector to every token and combining each token with that vector produce equivalent average movement when nearby verbs and objects are reliable cues; a difference appears when those cues are degraded. Finally, memory ablations show that label-derived summaries improve continuation, whereas mean pooling and two k -means centers do not surpass Token-Time.

Keywords: Temporal semantics · Semantic change · Transformer models · Traceability · Diagnostic evaluation

1 Introduction

Words do not preserve fixed meanings across time. In 1900, *broadcast* referred to scattering seeds across a field; today it describes a social media post reaching millions. Lexical semantic change detection (LSCD) usually learns vector representations from word co-occurrence in a separate corpus for each period and then compares them [10, 11]. This approach has produced important methods and benchmarks, but it summarizes change as one similarity score between separately trained models: `broadcast@1900` and `broadcast@2020` come from different models and become comparable only after their spaces are aligned, rather than being explicit queries to one model.

We call a representation *temporally traceable* if it provides an explicit vector for a word at a requested period in a common space and changes in its nearest neighbors across periods reflect changes in how the word is used. Consider *gay*, which in the early twentieth century was mainly associated with *cheerful* and *merry*, and by the late twentieth century with *queer* and *lesbian* [5]. Comparable representations can support socially relevant questions, including whether a word moved toward or away from stigmatized terms. Answering such questions requires directly comparable representations for *gay@1920* and *gay@1980*.

In this study, we test temporal traceability in a controlled environment where the semantic path is predefined. In natural corpora, an apparent change may instead reflect differences in word frequency, genre, or corpus composition. It is therefore difficult to tell whether a change in the learned vectors comes from the model architecture or the data. We introduce a corpus in which words follow predefined stable, drifting, or bifurcating trajectories. Four diagnostics test whether their neighbors follow these paths, rather than merely whether a classifier can recover a context label from their representations.

Against this suite we evaluate one non-temporal baseline (Standard) and three time-aware architectures. Additive gives every token the same period vector, Token-Time combines each token with that vector, and Memory-Augmented also summarizes earlier periods. We ask whether these designs improve temporal traceability over the non-temporal baseline and what each design contributes.

The paper makes four contributions. (1) We define temporal traceability as the ability to query an explicit word-at-period vector and test whether changes in its neighborhood reflect changes in the word’s use, rather than merely recover a label from its representation. (2) We introduce a controlled corpus with stable, drifting, and bifurcating trajectories and four diagnostics that separate neighbor movement from label prediction. (3) We show that all three time-aware architectures more than double the movement of drifting words relative to the non-temporal baseline. Additive and Token-Time produce equivalent average movement when contextual cues are reliable, while Token-Time performs better on the ambiguous classifier when they are not. (4) Memory ablations show that label-derived summaries improve continuation, whereas learned averages and two k -means centers do not surpass Token-Time.

2 Related Work

Lexical semantic change detection (LSCD) is the dominant computational framing for meaning shift. Diachronic word embeddings [5] learn one vector space per time slice, align the separately trained spaces, and measure how a word’s neighbors change. We share the view that meaning change should alter these neighbors, but use a different architecture: LSCD produces a score comparing two spaces rather than one model that can represent *token@time* directly. Standard benchmarks formalize this task [10, 11], but changes in frequency and genre can also affect their scores; Spanish shared tasks show that manual annotation is required even for modern corpora [13]. Our controlled corpus makes each se-

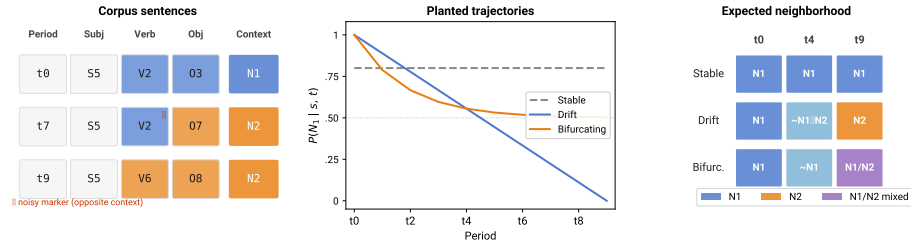


Fig. 1. Controlled diagnostic suite overview. *Left:* SVO sentences for a drifting subject across three periods; an N_1 verb appearing in an N_2 sentence at $t7$ (marked *) illustrates the deliberate noise in contextual cues. *Center:* predefined $P(N_1 | s, t)$ trajectories for Stable (flat), Drift (monotonic), and Bifurcating (levelling at 0.5) subjects across $t0$ – $t9$. *Right:* nearest-neighbor behavior expected from the predefined trajectory.

mantic path known, allowing differences to be attributed to model design rather than corpus composition.

Temporal neural language models avoid separate spaces by conditioning one model on time. Prior work uses timestamps for knowledge-intensive QA, masked language modeling, or period-stratified downstream evaluation [3, 9, 8]. These results show that providing time can improve predictions for the requested period, but do not test whether a word can be queried at different periods and its neighborhoods inspected and compared directly.

Finally, semantic change may involve bifurcation rather than monotonic drift. Giulianelli et al. show that words can develop coexisting senses [4]; a mean-pooled summary can blur their distinction by falling between them. Our stable, drifting, and bifurcating trajectories test whether one averaged vector per subject and period is adequate, and the memory ablations examine alternatives.

3 Controlled Diagnostic Suite

We construct a corpus of subject–verb–object (SVO) sentences spanning ten periods. Every sentence consists of one subject, one verb, and one object. We divide the 8 verbs and 8 objects into two groups, N_1 and N_2 , which act as distinct semantic contexts: N_1 uses $\{V1, \dots, V4\}$ and $\{O1, \dots, O4\}$; N_2 uses $\{V5, \dots, V8\}$ and $\{O5, \dots, O8\}$. A sentence generated from N_1 therefore looks like S5 V2 O3, while one from N_2 looks like S5 V6 O8; the subject token is the same in both cases, but its co-occurring words differ. For each of the 30 subjects, $P(N_1 | s, t)$ is the probability that a sentence containing subject s at period t is drawn from N_1 ; its evolution from $t0$ through $t9$ defines the subject’s predefined trajectory.

Figure 1 (center) shows the three predefined trajectory classes. Stable subjects maintain an approximately constant $P(N_1 | s, t)$ throughout. Drift subjects follow a monotonic trajectory from near-exclusive N_1 ($P(N_1 | s, t) \approx 1$ at $t0$) to near-exclusive N_2 ($P(N_1 | s, t) \approx 0$ at $t9$): a drifting subject that appears in

sentences like **S5 V2 O3** early on will appear in sentences like **S5 V6 O8** by the final period. Bifurcating subjects begin in N_1 and move toward a mixed state ($P(N_1 | s, t) \approx 0.5$) where both neighborhoods appear with equal frequency, approximating two coexisting senses. The suite contains 10 subjects per class.

Each verb and object is drawn from the opposite context group 25% of the time (Figure 1, left), so the model cannot recover the intended context from the accompanying words alone; it must use subject identity together with the period. The `true_context` label is withheld from encoder training and used only by the diagnostics.

This design separates differences caused by the architecture from differences in frequency, genre, or corpus composition. Because $P(N_1 | s, t)$ is known, we can test whether a model’s nearest neighbors follow the process that generated the corpus. The scope of these controlled diagnostics is revisited in the Discussion.

The experiments use four data conditions. The standard and ambiguous conditions vary the reliability of the accompanying words. In the standard condition, after N_1 or N_2 is selected to generate a sentence, each verb and object is drawn from that same group with 75% probability, matching the distribution used for masked language modeling (MLM) training. In the ambiguous condition, this probability is reduced to 50%, so the verb and object do not reveal which group generated the sentence on average. The contrastive condition presents the same masked-verb subject-object frame with two period labels and measures whether changing only the period changes the preferred context. The continuation condition contains periods $t8$ – $t9$, which are excluded from MLM training on $t0$ – $t7$.

4 Time-Aware Architectures

We evaluate one non-temporal baseline and three time-aware architectures, all sharing the same Transformer encoder backbone. Standard (no period input) is the non-temporal baseline. The three time-aware architectures differ in how they introduce time: Additive gives every token the same period vector; Token-Time combines each token with that vector before the encoder; and Memory-Augmented also lets the subject consult summaries of earlier periods.

All models follow the same pipeline based on the Transformer encoder [12]. A sentence is tokenized as `[CLS] S V O [SEP]`. Each token w_i is mapped to an input embedding $z_i \in \mathbb{R}^d$. The sequence $(z_1, \dots, z_n)$ is processed by a Transformer encoder, and the final hidden state at the subject position,

$$(h_1, \dots, h_n) = \text{Encoder}(z_1, \dots, z_n), \quad h_s = h_{\text{subj}},$$

is the `token@time` representation inspected by the diagnostics. All models are trained with MLM [2]: some tokens are masked, and the model reconstructs them from the remaining tokens and, when available, the period input. The variants differ in how they inject temporal information before the encoder.

The three time-aware models use a shared period vector $\tau(t) \in \mathbb{R}^d$, a learned projection of sinusoidal features of t/T , where $T = 10$ is the number of corpus periods. It is distinct from the position encoding p_i , which marks subject, verb,



Fig. 2. Controlled design comparison. **Standard:** Transformer baseline with no temporal signal. **Additive:** applies the same period vector to every token. **Token-Time:** Token-Time Transformer using joint token-time projection. **Memory-Augmented:** extension of Token-Time with past-period summary memory and temporal attention. New components are highlighted in yellow; dashed grey boxes mark inactive components.

or object position. Additive and Token-Time differ in how they combine $\tau(t)$ with token identity; Memory-Augmented lets the subject embedding consult vectors from earlier periods. Figure 2 summarizes the comparison.

4.1 Baseline: Standard Transformer

Standard is the null hypothesis: no period signal is available, so any temporal behavior must arise from changes in the observed words alone.

$$z_i = e(w_i) + p_i.$$

4.2 Additive Time-Conditioned Transformer

Additive tests the simplest use of time: adding the same period vector $\tau(t)$ to every token:

$$z_i = e(w_i) + p_i + \tau(t).$$

S5 V2 03 therefore receives different inputs at $t0$ and $t9$: each token is combined with $\tau(0)$ or $\tau(9)$. The same period vector is added at every position, so time enters each token in the same way.

4.3 Token-Time Transformer

Token-Time tests a learned alternative to direct addition. It concatenates each token embedding with $\tau(t)$ and projects the resulting $2d$ -dimensional vector back to d dimensions:

$$z_i = W \cdot \text{concat}(e(w_i), \tau(t)) + p_i,$$

where concat joins the two d -dimensional vectors into a $2d$ -dimensional vector, and the learned projection matrix $W \in \mathbb{R}^{d \times 2d}$ maps it back to d dimensions. The comparison with Additive tests whether this projection helps beyond adding the period vector directly.

4.4 Memory-Augmented Extension

The Memory-Augmented extension tests whether representations from earlier periods add useful information beyond the current token-time embedding. It begins with the same input embeddings as Token-Time. For a sentence containing S5 at $t7$, it builds embeddings for [CLS] S5 V 0 [SEP] and modifies only the subject embedding before the Transformer encoder sees the sentence.

The memory stores one past-period summary vector (often called a prototype) $m(s, t)$ for each subject and period. This vector is the mean of the final hidden states h_s from MLM training sentences containing subject s at period t . When processing a sentence at period t , the model can access only summary vectors from earlier periods. At $t7$, for example, it can use the S5 vectors from $t0$ through $t6$, but not from $t7$ or later.

The model uses attention to combine these past vectors. The current subject embedding z_s is the query, while M_s , with one past-period vector per row, provides the keys and values:

$$r_s = \text{Attn}(z_s, M_s, M_s).$$

The resulting vector r_s is a weighted combination of the past-period vectors. A learned scalar gate g controls how much this summary changes the current subject embedding:

$$\tilde{z}_s = z_s + g \cdot (\text{LN}(z_s + r_s) - z_s).$$

Here, LN denotes layer normalization. Because g is initialized at zero, the extension initially behaves exactly like Token-Time. Finally, $\tilde{z}_s$ replaces z_s in the input sequence. Verb and object embeddings are unchanged, so the subject is the only token that enters the encoder with information from earlier periods.

5 Evaluation Protocol and Diagnostics

We evaluate h_s with four diagnostics. Across these diagnostics, we compare each time-aware model with Standard, Token-Time with Additive, and Memory-

Augmented with Token-Time. These comparisons measure, respectively, the effects of providing time, using a learned projection instead of direct addition, and consulting earlier representations.

D1: Linear classifier [1]. D1 tests whether a linear classifier can tell from h_s whether the sentence was generated from N_1 or N_2 . We freeze the encoder and train the classifier on representations from the standard condition. Its standard-condition accuracy is in-sample; the ambiguous condition tests the same classifier without retraining it. High accuracy does not show how the representation’s nearest neighbors change across periods.

D2: Context drift score. For each evaluation sentence, we use h_s , the final representation of its subject, as the query. For example, for **S12 V2 07** at t_0 , the query is the vector at the **S12** position. We compare it with the subject representations from all other evaluation sentences and retrieve the $k = 10$ nearest by cosine similarity. The score is the proportion of neighbors whose **true_context** is N_1 : if 8 of the 10 neighbors come from N_1 , the score is 0.8. We average this score by trajectory class and period. The reported Δ is the score at t_9 minus the score at t_0 ; for example, a decrease from 0.8 to 0.2 gives $\Delta = -0.6$, indicating movement from N_1 toward N_2 . We also compare the per-period scores with the predefined $P(N_1 | s, t)$ trajectories.

D3: Contrastive test. D3 asks whether changing only the period changes the masked-verb prediction in the correct direction. Each of the 29 pairs contains the same subject–object frame at an early and a late period. For example, the model receives **S12 [MASK] 02** at both t_0 and t_9 . At each period, we sum the predicted probabilities for the N_1 verbs (**V1–V4**) and for the N_2 verbs (**V5–V8**). The pair counts as a correct sign flip if the N_1 total is higher at t_0 and the N_2 total is higher at t_9 . A sign-flip rate of 0.3 means that 30% of the pairs meet both conditions.

D4: Continuation diagnostic. D4 trains a separate linear classifier on standard-condition representations from t_0 – t_7 and evaluates it without retraining on continuation sentences from t_8 – t_9 . For example, for **S12 V6 07** at t_8 , the classifier receives the vector at the **S12** position and should identify N_2 as the group that generated the sentence. Accuracy is the proportion of correct N_1/N_2 predictions.

Auxiliary layer-wise comparison. We also compute linear centered kernel alignment (CKA) between models on the same standard held-out sentences, at the input and after each encoder layer. CKA compares their representation patterns; higher values indicate greater similarity [6].

Memory ablation. For Memory-Augmented, we also test an oracle version that replaces each learned summary vector with one built from the known context labels. For example, if 80% of a subject’s training sentences at a given period come from N_1 , its oracle summary is $0.8v_1 + 0.2v_2$, where the fixed orthogonal vectors v_1 and v_2 represent N_1 and N_2 . The attention and gate are unchanged. Because the ordinary model does not receive these labels, the oracle serves only as an analysis tool.

Implementation details. All models share the same Transformer encoder: $d = 64$, 4 attention heads, 2 layers, feed-forward width 128, dropout 0.1. The period encoding projects 32 sinusoidal features of t/T to d via a two-layer MLP. Training uses AdamW (lr = 10^{-3} , weight decay = 10^{-2}) with a cosine annealing schedule, 30 epochs, batch size 64. The corpus contains 5,000 SVO sentences; MLM training uses 3,159 sentences from periods $t0$ – $t7$; 763 sentences form the standard held-out set used for D1–D2 and to train the D1 classifier; the 615 sentences from $t0$ – $t7$ also train the separate D4 classifier. Another 794 sentences from $t8$ – $t9$ are reserved for D4 and excluded from both MLM and classifier training. Validation sentences from all ten periods are used to select the version of each model evaluated in the diagnostics. Thus, D4 tests whether a classifier trained on $t0$ – $t7$ transfers to later sentences; it is not a fully held-out forecast because model selection uses validation data from $t8$ – $t9$.

6 Results

We report means and 95% confidence intervals over 31 training seeds. Period input creates the clearest separation across diagnostics; differences among the time-aware designs are smaller and condition-dependent. For drifting subjects, all three time-aware models reach $\Delta \approx -0.56$, more than twice Standard’s -0.221 .

6.1 Changes in Nearest Neighbors

Figure 3 and Table 1 show how the proportion of N_1 neighbors changes across periods. For drifting subjects, Standard decreases by 0.221, with a 95% CI of $[-0.240, -0.203]$. Token-Time decreases by 0.563, more than twice as much. Additive produces a similar decrease of 0.567.

Stable subjects also shift toward N_2 , with Δ between -0.150 and -0.180 . This residual movement is about one-third of the change for drifting subjects ($\Delta \approx -0.56$), showing strong trajectory sensitivity but incomplete selectivity. Bifurcating subjects fall between these cases: the time-aware models reach values near 0.53 at $t9$, close to their predefined 50–50 mixture.

We used a paired equivalence test to determine whether the Additive and Token-Time scores are sufficiently close, rather than merely failing to find a difference. We treated absolute differences below 0.05 as equivalent. The mean paired difference (Additive minus Token-Time) is -0.003 , with a 90% CI of $[-0.026, +0.019]$; the TOST supports equivalence ($p_{\text{TOST}} < 0.001$). Equivalence also holds with the narrower margin of 0.034 ($p_{\text{TOST}} = 0.014$).

6.2 Classifier and Contrastive Diagnostics

The linear classifier reveals a difference that the drift score does not. In the standard condition, all time-aware models score above Standard (0.875): Additive reaches 0.892, while Token-Time and Memory-Augmented both reach 0.881.

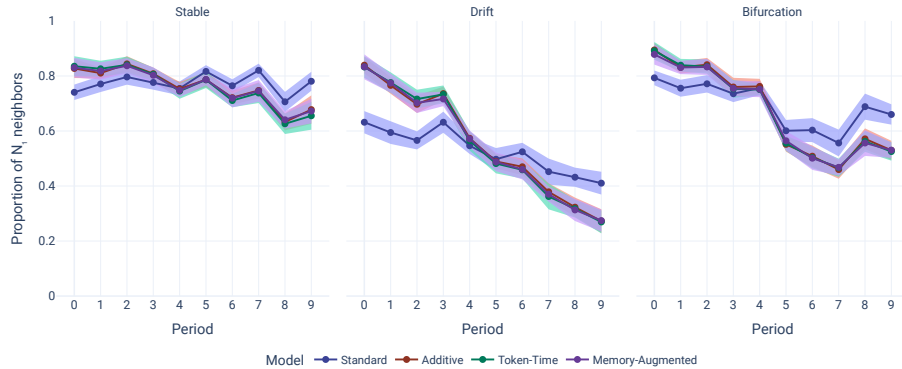


Fig. 3. Context drift score across periods. All time-aware models move drifting subjects from N_1 toward N_2 more than Standard; Additive, Token-Time, and Memory-Augmented produce similar trajectories. Stable subjects move less, and bifurcating subjects end between N_1 and N_2 .

Table 1. Context drift score at the first and last periods. Values are means over 31 seeds; $\Delta = t_9 - t_0$; 95% CI half-widths in brackets.

Class	Model	t_0	t_9	Δ	[CI $_{\pm}$]
Stable	Standard	0.741	0.781	+0.040	[± 0.017]
	Additive	0.827	0.678	-0.150	[± 0.027]
	Token-Time	0.835	0.655	-0.180	[± 0.024]
	Mem-Aug.	0.830	0.672	-0.158	[± 0.021]
Drift	Standard	0.632	0.410	-0.221	[± 0.018]
	Additive	0.839	0.272	-0.567	[± 0.019]
	Token-Time	0.833	0.269	-0.563	[± 0.020]
	Mem-Aug.	0.833	0.274	-0.559	[± 0.022]
Bifurc.	Standard	0.793	0.660	-0.133	[± 0.018]
	Additive	0.895	0.530	-0.365	[± 0.017]
	Token-Time	0.891	0.525	-0.366	[± 0.016]
	Mem-Aug.	0.878	0.530	-0.348	[± 0.015]

The ordering changes when verbs and objects provide no reliable context clue: Standard reaches 0.565, Additive 0.579, Token-Time 0.595, and Memory-Augmented 0.598. Token-Time is 1.6 percentage points above Additive, and their 95% confidence intervals do not overlap.

Classifier accuracy cannot establish temporal traceability by itself: it tests whether the predefined context can be predicted, not whether changes in the word’s neighbors follow changes in its use across periods.

All time-aware models produce sign flips at rates from 0.587 to 0.624; Standard scores 0.000 without period input. Their confidence intervals overlap, so D3 does not distinguish among the temporal variants.

In D4, all time-aware models score above Standard (0.717): Additive reaches 0.746, Token-Time 0.748, and Memory-Augmented 0.739 (Table 2). Additive and Token-Time thus behave similarly in D2 and D4; Token-Time’s advantage appears only when contextual words are uninformative.

Table 2. Main results (means, 31 seeds). Drift: context drift score Δ for drifting subjects; Flip: contrastive sign-flip rate (D3); Ambig./Cont.: linear-classifier accuracy on h_s . 95% CI in brackets.

Model	Drift	Flip	Ambig.	Cont.
Standard	-0.221 $[\pm 0.018]$	0.000	0.565 $[\pm 0.004]$	0.717 $[\pm 0.004]$
Additive	-0.567 $[\pm 0.019]$	0.602 $[\pm 0.053]$	0.579 $[\pm 0.005]$	0.746 $[\pm 0.007]$
Token-Time	-0.563 $[\pm 0.020]$	0.624 $[\pm 0.048]$	0.595 $[\pm 0.006]$	0.748 $[\pm 0.005]$
Mem-Aug.	-0.559 $[\pm 0.022]$	0.587 $[\pm 0.051]$	0.598 $[\pm 0.008]$	0.739 $[\pm 0.008]$

6.3 Memory Ablations

Memory-Augmented remains above Standard on continuation (0.739 vs. 0.717) but not Token-Time (0.748); the paired difference is $-0.009 [\pm 0.010]$. The oracle shows that the mechanism can use informative history: label-derived vectors raise continuation accuracy to 0.762 $[\pm 0.006]$, 0.013 above Token-Time $[\pm 0.009]$. Its sign-flip rate is 0.694 $[\pm 0.118]$, 0.070 above Token-Time, although this estimate has a wide confidence interval.

Table 3. Memory ablations (means, 31 seeds). Continuation (D4): accuracy of a classifier trained on $t0$ – $t7$ and evaluated on separate $t8$ – $t9$ sentences; Flip: contrastive sign-flip rate (D3); 95% CI half-widths in brackets.

Model	Cont.	Flip	Δ cont. vs. Token-Time
Token-Time (ref.)	0.748 $[\pm 0.005]$	0.624 $[\pm 0.048]$	–
Mem-Aug. learned	0.739 $[\pm 0.008]$	0.587 $[\pm 0.051]$	$-0.009 [\pm 0.010]$
Mem-Aug. oracle	0.762 $[\pm 0.006]$	0.694 $[\pm 0.118]$	$+0.013 [\pm 0.009]$

To test whether a single average discards a possible two-group structure, we replaced it with two k -means cluster centers per subject and period. The rest of the memory architecture remains unchanged. Across 15 matched seeds, this alternative does not improve continuation accuracy or the sign-flip rate over either mean pooling or Token-Time; all four confidence intervals for the differences include zero.

7 Discussion and Conclusion

Temporal conditioning more than doubles neighbor movement for drifting subjects relative to Standard, and this movement is about three times the residual shift observed for stable subjects. Under reliable contextual cues, direct addition is as effective as a learned token–time interaction, making Additive the more parsimonious design in this setting.

Additive and Token-Time are equivalent when verbs and objects provide reliable context clues (TOST, $\delta = 0.05$, $p_{\text{TOST}} < 0.001$). When those words provide no reliable clue, the classifier gives Token-Time a small advantage over Additive (0.595 vs. 0.579). The layer-wise CKA comparison tells a complementary story:

Additive-Token-Time similarity rises from 0.726 at the input to 0.887 after the encoder. Their overall representation patterns become more similar with depth, while the ambiguous classifier still detects a condition-specific difference.

If ambiguity systematically favored Token-Time, its advantage should grow as contextual cues become less reliable. The 15-seed noise sweep does not show this pattern. For D2, the slope is $+0.093$ (95% CI $[-0.279, +0.464]$, $d_z = 0.14$, $p = 0.601$); for classifier accuracy, it is -0.019 (95% CI $[-0.105, +0.067]$, $d_z = -0.12$, $p = 0.644$). The ambiguous-condition result is therefore evidence for a specific condition, not general Token-Time superiority.

The memory ablations further narrow the design space. The oracle shows that the existing attention and gate can exploit informative history, while neither one learned average nor two unsupervised k -means cluster centers surpass Token-Time. This suggests that the bottleneck lies in the information recovered by these unsupervised summaries, rather than in mean pooling alone.

These results position the suite as a controlled architectural audit before evaluation on natural LSCD data. RuShiftEval, for example, follows the same words across three periods and includes change followed by stability as well as continuous change [7]. With annotated targets, stable words can serve as negative controls, while gradual change and the gain or coexistence of senses provide approximate counterparts to our drifting and bifurcating trajectories.

A natural-corpus study can test whether the D2 equivalence persists when local contexts identify the current sense and whether Token-Time gains an advantage for more ambiguous usages. Such a study must adapt the diagnostics to imperfectly annotated trajectories and account for open-ended polysemy, metaphor, register variation, frequency shifts, and genre differences, none of which the controlled SVO corpus reproduces.

In this controlled setting, direct addition matches more complex temporal conditioning in average neighbor movement. Token-Time’s localized advantage under ambiguity identifies where a learned interaction may matter, while the memory ablations show that useful history can help when its summary preserves the relevant distinction. More broadly, period or context-label recovery is not enough: temporal traceability requires explicit word-at-period representations whose neighborhoods can be inspected across time and whose changes reflect changes in how the word is used.

References

1. Conneau, A., Kruszewski, G., Lample, G., Barrault, L., Baroni, M.: What you can cram into a single $\text{\$}\&\text{!}\#\text{^*}$ vector: Probing sentence embeddings for linguistic properties. In: Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL). pp. 2126–2136 (2018)
2. Devlin, J., Chang, M.W., Lee, K., Toutanova, K.: BERT: Pre-training of deep bidirectional transformers for language understanding. In: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL). pp. 4171–4186 (2019)

3. Dhingra, B., Cole, J.R., Eisenschlos, J.M., Gillick, D., Eisenstein, J., Cohen, W.W.: Time-aware language models as temporal knowledge bases. *Transactions of the Association for Computational Linguistics (TACL)* **10**, 257–273 (2022)
4. Giulianelli, M., Del Tredici, M., Fernández, R.: Analysing lexical semantic change with contextualized word representations. In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*. pp. 3960–3973 (2020)
5. Hamilton, W.L., Leskovec, J., Jurafsky, D.: Diachronic word embeddings reveal statistical laws of semantic change. In: *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*. pp. 1489–1501 (2016)
6. Kornblith, S., Norouzi, M., Lee, H., Hinton, G.: Similarity of neural network representations revisited. In: *Proceedings of the 36th International Conference on Machine Learning. Proceedings of Machine Learning Research*, vol. 97, pp. 3519–3529. PMLR (2019)
7. Kutuzov, A., Pivovarova, L.: Three-part diachronic semantic change dataset for Russian. In: *Proceedings of the 2nd International Workshop on Computational Approaches to Historical Language Change*. pp. 7–13. Association for Computational Linguistics (2021). <https://doi.org/10.18653/v1/2021.lchange-1.2>
8. Loureiro, D., Barbieri, F., Neves, L., Anke, L.E., Camacho-Collados, J.: TimeLMs: Diachronic language models from Twitter. In: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL): System Demonstrations*. pp. 251–260 (2022)
9. Rosin, G., Radinsky, K.: Time masking for temporal language models. In: *Proceedings of the 15th ACM International Conference on Web Search and Data Mining (WSDM)*. pp. 833–841 (2022)
10. Schlechtweg, D., McGillivray, B., Hengchen, S., Dubossarsky, H., Tahmasebi, N.: SemEval-2020 task 1: Unsupervised lexical semantic change detection. In: *Proceedings of the 14th International Workshop on Semantic Evaluation (SemEval)*. pp. 1–23 (2020)
11. Tahmasebi, N., Borin, L., Jatowt, A.: Survey of computational approaches to lexical semantic change. *Computational Approaches to Semantic Change* pp. 1–91 (2021)
12. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. In: *Advances in Neural Information Processing Systems (NeurIPS)*. vol. 30, pp. 5998–6008 (2017)
13. Zamora-Reina, F.D., Bravo-Marquez, F., Schlechtweg, D.: LSCDiscovery: A shared task on semantic change discovery and detection in Spanish. In: *Proceedings of the 3rd Workshop on Computational Approaches to Historical Language Change (LChange)*. pp. 149–164 (2022)